

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3041 - REDES

SECCIÓN – 20

Ing. Jorge Yass



Laboratorio 3: Protocolos de enrutamiento

Javier André Benítez García 23405

Pedro Ruben Ávila Cofiño 23089

GUATEMALA, 13 de agosto de 2026

Curso	CC3067 Redes — Universidad del Valle de Guatemala
Laboratorio	3 — Algoritmos de Enrutamiento
Algoritmo	Link State (Dijkstra)
Lenguaje	Zig 0.16
Repositorio	link_state_routing
Integrantes	Javier André Benitez García · _____
Topología	3 parejas / 6 nodos

1. Requisitos e instalación

Zig 0.16.0 es la única herramienta necesaria. Alternativa reproducible: nix develop, que deja zig, zls y lldb listos con el flake.nix del repositorio. La única dependencia del proyecto es zig-cli, que zig build descarga automáticamente.

```
git clone <repo> && cd link_state_routing
zig build          # binario en zig-out/bin/link_state_routing
zig build test     # 11 pruebas
```

No hay pasos adicionales de instalación: el binario resultante se ejecuta igual en macOS y en Linux.

2. Ejecución

Un solo binario con dos planos, seleccionados con --plane_type. El plano de control converge, escribe nodo_tabla_enrutamiento.csv y termina; el plano de datos lee ese archivo y reenvía los mensajes que le llegan.

```
# Plano de control
zig-out/bin/link_state_routing --plane_type Control \
  --host 127.0.0.1 --port 5051 \
  --nodes_neighbors_path data/neighbors_5051.txt \
  --routing_table_path nodo_tabla_enrutamiento.csv

# Plano de datos
zig-out/bin/link_state_routing --plane_type Data \
  --host 127.0.0.1 --port 5051 \
  --routing_table_path nodo_tabla_enrutamiento.csv

# Inyectar un mensaje y salir
zig-out/bin/link_state_routing --plane_type Data \
  --host 127.0.0.1 --port 5051 \
  --routing_table_path nodo_tabla_enrutamiento.csv \
  --send data/msg_auth.json

# Demo completo de los 6 nodos
./scripts/demo.sh
```

3. Protocolo implementado

JSON plano, un objeto por línea, según la propuesta de protocolo acordada entre las 3 parejas. El transporte es una conexión TCP por mensaje: connect, una línea JSON, close.

```

{"type":"HELLO","from":"<ip_origen>"}

{"type":"LSA","origin":"<ip_creador>","seq":<int>,"ttl":<int>,"links":{"<ip_vecino>":<costo>},"from":"<ip_emisor_actual>"}

{"type":"AUTH"|"WITHDRAW"|"ERROR"|"LOGOUT",
 "origin":"<ip_nodo_ATM>","destination":"<ip_nodo_servidor>","payload":{"  }}"

```

Manejo de ciclos en el flooding

- Si $\text{ttl} \leq 0$, se descarta.
- Si seq es menor o igual al último guardado para ese origen, se descarta y no se reenvía.
- En caso contrario: $\text{ttl} - 1$, from pasa a ser este nodo, y se reenvía a todos los vecinos excepto por donde llegó. Los campos origin y seq no se modifican.

Costo de enlace

RTT del paquete HELLO en milisegundos, con piso en 1. En loopback el RTT redondea a 0 y todas las rutas empatarían en costo 0, dejando a Dijkstra sin nada que distinguir.

Identidad de nodo

El identificador viaja en from, origin, destination y en las claves de links. Sobre Tailscale se usa la IP a secas (--id 100.64.0.7); en pruebas locales los 6 nodos comparten 127.0.0.1, por lo que el valor por defecto es host:puerto. La tabla guarda ip y puerto en columnas separadas, así que ambos esquemas funcionan sin cambiar código.

Archivos

Archivo de vecinos (una línea por vecino directo) y tabla de ruteo, que es el contrato entre el plano de control y el plano de datos:

```

vecinos: id,host,puerto      ó      host:puerto
tabla:   destino,siguiente_salto,costo,ip,puerto

```

4. Resultados

Topología de prueba

Anillo A–B–C–D–E–F–A con la cuerda B–E, en los puertos 5051 a 5056.

```

A(5051)  -----  B(5052)  -----  C(5053)
    |              |              |
    |              | (cuerda)     |
    |              |              |
F(5056)  -----  E(5055)  -----  D(5054)

```

Convergencia

Los 6 nodos alcanzaron una base de datos de estado de enlace completa (6 orígenes) y escribieron su tabla de enrutamiento.

Tabla obtenida — nodo A (127.0.0.1:5051)

```
destino,siguiente_salto,costo,ip,puerto
127.0.0.1:5052,127.0.0.1:5052,1,127.0.0.1,5052
127.0.0.1:5056,127.0.0.1:5056,1,127.0.0.1,5056
127.0.0.1:5053,127.0.0.1:5052,2,127.0.0.1,5052
127.0.0.1:5055,127.0.0.1:5056,2,127.0.0.1,5056
127.0.0.1:5054,127.0.0.1:5052,3,127.0.0.1,5052
```

Coincide con el cálculo hecho a mano: B y F a un salto; C vía B y E vía F a dos; D a tres por cualquiera de los dos lados del anillo. El nodo F usa la cuerda para alcanzar B con costo 2 en lugar de 3.

Entrega extremo a extremo

Mensaje AUTH inyectado en A (5051) con destino D (5054):

```
[5051] REENVIADO 127.0.0.1:5051 -> 127.0.0.1:5052 (destino final 127.0.0.1:5054)
[5052] REENVIADO 127.0.0.1:5052 -> 127.0.0.1:5055 (destino final 127.0.0.1:5054)
[5055] REENVIADO 127.0.0.1:5055 -> 127.0.0.1:5054 (destino final 127.0.0.1:5054)
[5054] ENTREGADO en 127.0.0.1:5054 | type=AUTH origin=127.0.0.1:5051
```

Solo el nodo destino entrega el mensaje; los intermedios únicamente lo reenvían, sin modificar el payload.

Conformidad del formato en el cable

Bytes capturados por un receptor externo conectado a un nodo real, para verificar la interoperabilidad con las otras parejas:

```
{"type": "HELLO", "from": "127.0.0.1:5098"}
{"type": "LSA", "origin": "127.0.0.1:5098", "seq": 1, "ttl": 8,
 "links": {"127.0.0.1:5099": 1}, "from": "127.0.0.1:5098"}
```

Nombres, orden y tipos coinciden con lo acordado: links es un objeto de claves dinámicas, seq incrementa en cada ronda y ttl sale en 8.

Pruebas automáticas

zig build test ejecuta 11 pruebas, todas correctas: serialización exacta de los formatos acordados, descarte por seq en la base de datos de estado de enlace, parseo del archivo de vecinos y Dijkstra, incluyendo un caso donde la ruta más corta no es el enlace directo.